ONBOARDING GUIDE · v1.0

The Limitless Stack

Seven tools, one operating system.
Designed to compound knowledge over time.

Open Scaffold Labs · github.com/Open-Scaffold-Labs/LimitlessStack

00 Welcome

What you'll have when you're done

Seven tools wired into one operating system: **Claude · CLAUDE.md · Obsidian · NotebookLM · Pinecone · Hub Workspace · Paperclip**. This guide takes you from *git clone* to a working stack in roughly 30–45 minutes — and explains how the system *compounds knowledge and self-heals over time*, so the version you run on day 90 is meaningfully smarter than the one you booted on day 1.

Read it once before you start. Paste it into your Claude session as the onboarding context so your Claude inherits the same operating discipline this stack runs on.

By the end of this PDF you'll have:

- A compounding knowledge base in Obsidian — every new source updates many pages, never derived from scratch.
- Semantic recall across every repo and document via Pinecone — natural-language search.
- Curated topic notebooks in NotebookLM, including a special *reminder bucket* your Claude reads at the start of every session.
- Mechanical session rituals — Roll Call preflight at start, 8-step checklist at end. Bundled as installable skills, enforced by your CLAUDE.md.
- A living anti-patterns log + a self-healing loop — autonomous diagnose-and-repair pipeline for code bugs, with CLAUDE.md as trust anchor.
- Six skills auto-installed to `~/.claude/skills/` by the installer.

01 The seven tools

Each plays one role. The protocol is in how they're connected.

Tool	Role	What it owns
Claude	Reasoning engine	Reads, writes, connects, decides
CLAUDE.md	Identity + rules	Trust anchor, voice, guardrails per context
Obsidian	Knowledge base	Structured wiki: architecture, decisions, entities
NotebookLM	Research desk	Deep research across curated source collections
Pinecone	Semantic memory	Full-text recall across every repo and document
Hub Workspace	Multi-model agent runtime	Chat surface — Gemini default, Claude opt-in
Paperclip	Coordination	Org chart, budgets, tickets, approvals

The three problems it solves

Amnesia. Every Claude session starts cold. Long contexts balloon costs and degrade reasoning. Classic RAG (chunks → answer → forget) doesn't compound. **Solved by** a persistent memory layer Claude can read AND write — wiki + Pinecone + NotebookLM.

Drift. Operating rules, anti-patterns, and recent decisions live in files. Without enforcement, agents stop reading them. **Solved by** three-layer enforcement: mandatory-first-action banner in CLAUDE.md, skill descriptions that auto-trigger on greetings, Roll Call preflight that mechanically gates substantive work.

Bug repair at scale. Many vertical apps on one shared architecture means manual triage doesn't scale. **Solved by** a self-healing pipeline: in-app capture → Claude diagnostic agent (CLAUDE.md as system prompt) → operator review → sandboxed repair agent → PR → human merge → log entry that future sessions learn from.

02 The flywheel

How the Stack compounds over time

The Limitless Stack isn't a static toolkit. It's a system designed to get *measurably smarter every time you use it.* Four feedback loops compound knowledge across the stack — they run in parallel and reinforce each other.

Loop 1 — Source ingest. Drop a source in `raw/`. Claude reads it, summarizes into `wiki/sources/`, extracts crisp factual claims, identifies entities and concepts. **What compounds:** the more sources you ingest, the more cross-references the next ingest can make. Your Claude is building a graph, not a list.

Loop 2 — Wiki refinement. Contradictions get flagged with `> [!warning]` callouts — never silently overwritten. Substantive answers file back as `wiki/synthesis/`. Periodic lint catches orphans, broken links, stale claims. **What compounds:** the wiki gets sharper, less contradictory, and more navigable over time.

Loop 3 — Index sync. Step 4 of end-of-session runs Pinecone sync; step 5 runs NotebookLM refresh; step 6 verifies it landed. Every wiki edit reflects in semantic search and topic notebooks within minutes. **What compounds:** Pinecone recall improves with every chunk; NotebookLM syntheses get richer with every source.

Loop 4 — Anti-patterns + reminder bucket. When a session catches a recurring failure, it lands as a numbered entry in `claude-anti-patterns.md`. That file is in the reminder bucket allowlist, so the next session reads the new entry as part of opening context. **What compounds:** mistakes never get repeated. The system learns from itself, session by session.

03 90 days of compounding

Concrete numbers, not abstract claims.

Abstract framing only goes so far. Here's what an actual operator's stack looks like at three checkpoints, drawn from real activity in the OpenScaffold vault.

DAY 1	DAY 30	DAY 90
Wiki pages **7**	Wiki pages **~80**	Wiki pages **~250**
Pinecone chunks **0**	Pinecone chunks **~3,500**	Pinecone chunks **~15,000**
NotebookLM buckets **2**	NotebookLM buckets **5**	NotebookLM buckets **6+**
Anti-patterns **11 (vendored)**	Anti-patterns **14 (+3 yours)**	Anti-patterns **20+**
Roll Call **READY**	Wrap-completion **~95%**	Ramp time **days, not weeks**

Day 1. Fresh install: vendored anti-patterns and foundational pages give you immediate institutional memory other operators paid for. Roll Call returns READY because everything you have is wired correctly.

Day 30. Each new source has averaged 5–7 wiki pages touched. Semantic queries return results scoring ≥ 0.85. Reminder bucket has been refreshed and verified 30 times. You've added 3 of your own anti-patterns; future sessions inherit your team's specific lessons.

Day 90. The Obsidian graph view shows clear domain clusters. Lint passes are short — the wiki self-corrects most contradictions during ingest now. Topic-specific synthesis queries return quality on par with manually-written architecture docs. New contributors ramp in days because the wiki + reminder bucket + skills give them everything an experienced operator would explain.

The compounding is measurable, not abstract. Page count, chunk count, anti-pattern count, query latency, ramp time. If your numbers aren't moving in the right direction over 90 days, the loop has a leak — usually a skipped end-of-session step. The verification protocol exists to catch those leaks early.

04 Self-healing

The fifth flywheel — autonomous bug repair.

Once your stack is wired up, give Claude permission to fix bugs autonomously — with CLAUDE.md as trust anchor, sandboxed execution, and human review before anything merges. The biggest force multiplier in the stack.

1. Capture. User hits a bug, clicks the floating bug icon. Description + automatic context bundle (console logs, network, recent actions, viewport) gets posted to the bug-report endpoint.

2. Diagnose. Server inserts the bug record. Background promise fires Claude with CLAUDE.md as *system prompt* and bug context as user message. CLAUDE.md is the trust anchor — domain rules, conventions, what's safe to touch.

3. Review. Admin view shows: severity, confidence, root cause, suspected files, proposed fix. Operator decides whether to dispatch the repair agent. Cheap diagnostics, expensive repair — separation of concerns.

4. Dispatch. GitHub `repository_dispatch` event with bug context fires the repair workflow. Operator approval gates this.

5. Init. GitHub Actions runner checks out repo, creates a branch, installs the agent SDK. Sandboxed — never on prod.

6. Investigate. Claude agent runs with a tool whitelist (read, list, search, edit, write, bash, finish). 25-turn budget. No infinite loops.

7. Commit. If the agent produced changes, stage and commit. If not, report failure with reasoning.

8. PR. Bot opens a PR with full context: original user description, diagnostic finding, agent investigation summary, link to workflow run.

9. Merge. Standard review process. Auto-merge OFF by default — every fix gets a human eyeball.

10. Reconcile. Bug report updated with full trajectory. Wiki gets a log entry. New failure patterns go into anti-patterns.md.

05 Principles

Five rules that make self-healing trustworthy.

1. **CLAUDE.md as trust anchor**

   Both diagnostic and repair agents receive CLAUDE.md as *system prompt*, never bypass it. It encodes 'don't touch the migrations directory without asking,' 'never modify the shared users table,' 'this app uses lsh_ table prefix.' The agent inherits the operator's full operational context.

2. **Diagnose before repair**

   Always run the cheap diagnostic pass first. Repair is optional and gated. Separates 'figure out what's wrong' (fast, low-risk, frequent) from 'actually change code' (slower, higher-risk, gated by operator approval).

3. **Sandboxed execution**

   The repair agent runs in an ephemeral GitHub Actions runner with a code-level tool whitelist — never on prod, never with broad shell access. The whitelist is enforced in the agent SDK, not just suggested in the prompt.

4. **Human merge by default**

   PRs, not direct commits. Auto-merge requires explicit per-repo policy. Operators stay in the loop on every fix.

5. **Closed-loop auditability**

   Every state transition is recorded on the bug-report record: dispatch, diagnostic, repair, PR, merge, post-merge log entry. Operators can audit the full trajectory and replay any step.

How self-healing compounds with the other loops

Each successful self-heal updates three things: the codebase (PR), the wiki/log (auto-appended on merge), and — if a new failure mode emerged — `claude-anti-patterns.md`. Next session's Roll Call sees the new commit, the new log entry, and the new anti-pattern via the reminder bucket. Future agents diagnosing similar bugs start with that context already loaded.

Self-healing is opt-in. First time you run the stack, self-heal is not active — you set up the GitHub Actions workflow, agent SDK, and bug-report endpoint per repo. The `self-heal/templates/` directory in the LimitlessStack repo has the canonical files (workflow YAML, agent script, setup guide) you copy into each app repo when you're ready. Start with one app, prove the loop, then roll out.

06 The six skills

Auto-installed by ./install.sh

Each skill is a self-contained Markdown file Claude reads on demand. The **description** field makes it auto-trigger — when Claude sees a phrase that matches, the skill loads and Claude follows its instructions. All six install to `~/.claude/skills/<name>/SKILL.md`.

Skill	What it does	Triggers on
`limitless-stack`	Umbrella protocol. 4-tool lookup, mandatory first action, end-of-session checklist, reminder bucket, anti-patterns, code discipline.	References to Limitless Stack, OpenScaffold work, the seven-tool protocol.
`roll-call`	Session-start preflight. Calls limitless-preflight.sh. Verifies all 7 tools. Returns READY/WARN/BLOCK.	"hey claude", "are you ready", "roll call", "preflight", session-opening greetings.
`notebooklm`	Full NotebookLM CLI API. Notebooks, sources, queries, artifacts (audio overviews, study guides, briefings, mind maps).	References to NotebookLM, requests to create notebooks or generate artifacts.
`four-tool-lookup`	Codifies the wiki → Pinecone → NotebookLM lookup discipline.	Factual or architectural questions about the vault's domain.
`verify-before-claim`	Guards against false 'unavailable' claims. Forces verification across execution paths.	Claude is about to declare any tool, resource, or service "unavailable".
`karpathy-guidelines`	Code-edit discipline. 4 named principles: Think Before Coding, Simplicity First, Surgical Changes, Goal-Driven Execution. MIT mirror.	Writing, reviewing, or refactoring code.

Operational tools shipped alongside (in `tools/`)

`limitless-preflight.sh`
The script Roll Call calls. Validates 7 tools, dedupe-sweeps every NotebookLM bucket, returns 0/1/2.

`session-bootstrap.sh`
Current-thesis snapshot at session start (wiki state, Pinecone count, recent log entries).

`pinecone-sync.py`
Chunks corpus, upserts to index. `--changed-only`, `--dry-run`, `--repo NAME`.

`notebooklm-wiki-refresh.py`
Routes wiki files into per-project + default + reminder buckets. Verifies content actually landed.

`notebooklm-dedupe.py`
Sweeps any bucket for duplicates.

`pinecone-search.py`
Semantic search with optional `--repo` filter.

07 Enforcement

Three layers that make rituals reliable.

There's no single magic hook. Three layers work together — understanding all three is the difference between Claude following the protocol by accident and Claude following it reliably.

Layer 1 — CLAUDE.md mandatory-first-action banner

Your vault's CLAUDE.md (seeded by install.sh) opens with this banner:

```
■ MANDATORY FIRST ACTION — NO EXCEPTIONS
Before answering ANY question, making ANY file changes, or starting ANY task:
  1. Invoke the roll-call skill.
  2. Run tools/session-bootstrap.sh.
  3. Query the reminder NotebookLM bucket.
  4. Read wiki/index.md.
  5. Do NOT skip these steps.
```

Claude Code and Cowork read CLAUDE.md automatically at session start. The banner is the first thing in the file, with explicit "NO EXCEPTIONS" framing. Claude treats it as a hard precondition.

Layer 2 — skill descriptions auto-trigger

The roll-call skill description specifically lists trigger phrases: "hey claude", "are you ready", "roll call", "preflight", "is everything connected". Even if Claude blew past the CLAUDE.md banner (it won't), the moment you say a greeting the skill loads and Claude follows its instructions.

Layer 3 — explicit bootstrap prompt (manual backup)

Belt and suspenders. Paste this when you want to be 100% sure:

```
Hey Claude. This vault uses the Limitless Stack protocol. Run roll-call,
then bootstrap, then query my reminder NotebookLM bucket for the current
operating rules and recent anti-patterns. Then read wiki/index.md.
Then we'll start work.
```

End-of-session — same three-layer model

The 8-step checklist (in CLAUDE.md and limitless-stack SKILL.md) is enforced the same way: banner, skill, explicit prompt. Most reliable phrase: *"Run the end-of-session checklist."* Claude walks through all eight steps, runs the scripts, reports verify_failed counts.

Pro tip: next session's Roll Call catches most failures from the previous session's wrap. If Step 1 (commit + push) was skipped, Roll Call's git-status check fires WARN. The two rituals are designed to validate each other.

08 Install

Five steps. Each with a verification check.

Accounts: GitHub (org access), Pinecone (free tier OK), Google (NotebookLM access), invite to LimitlessStack repo.
Software: Homebrew, Python 3.11, Node.js, Obsidian, gh CLI, Chrome with the Claude extension.

1 Clone and install

```
$ gh repo clone Open-Scaffold-Labs/LimitlessStack ~/LimitlessStack
$ cd ~/LimitlessStack
$ ./install.sh ~/your-vault-path
```

Verify: `ls ~/.claude/skills/` shows six skills. `ls ~/your-vault-path/wiki/` shows index.md, log.md, overview.md, plus four foundational pages.

2 Pinecone — Keychain + first sync

```
$ security add-generic-password -a pinecone -s pinecone-api-key -U -w
  # (paste key when prompted)

$ cd ~/your-vault-path
$ python3.11 tools/pinecone-sync.py
```

Verify: `python3.11 tools/pinecone-search.py "any test query"` returns results.

3 Create your NotebookLM buckets

In Chrome (signed into NotebookLM), create at minimum two notebooks: **Wiki default** and **Reminder bucket**. Note each notebook's ID.

Per-project notebooks are optional but recommended.

4 Wire bucket IDs into the config

Edit `tools/notebooklm-wiki-refresh.py` — replace the example bucket IDs (`NOTEBOOK_ROUTES`, `DEFAULT_ROUTE`, `REMINDER_NOTEBOOK_ID`, `REMINDER_FILES`) with yours. Same for any bucket-ID block in `tools/limitless-preflight.sh`.

5 Customize for your domain + run Roll Call

```
Edit ~/your-vault-path/CLAUDE.md — replace [YOUR DOMAIN] placeholders. Then verify:

$ bash tools/limitless-preflight.sh ; echo "exit: $?"
```

Exit 0 = READY · 1 = WARN · 2 = BLOCK.

09 Operating

Day-to-day moves that keep the loop alive.

Ingest a new source. Drop in `raw/`. Ask: *"Ingest the new source at raw/{filename}."* Claude reads, creates a source page, updates entity/concept pages, flags contradictions, updates index, appends log entry.

Ask a question. Just ask. Four-tool lookup runs automatically. Substantive answers get filed back as `wiki/synthesis/<slug>.md`.

Periodic lint. Every few weeks: *"Run a lint pass and write the report."* Surfaces contradictions, stale claims, orphans, broken links, coverage gaps, index drift.

Catch a new failure mode. Append numbered entry to `wiki/synthesis/claude-anti-patterns.md`. Format: trigger pattern, why-it's-tempting, why-it's-wrong, corrective rule. Future sessions read it via the reminder bucket.

Schedule nightly Pinecone sync. `crontab -e` running `tools/pinecone-sync.py --changed-only` overnight. Keeps the index current without ceremony.

Customization checklist

When you clone, you're getting the worked example. To make it yours:

s in `otebooklm-wiki-refresh.py` + `imitless-preflight.sh`. al, must change.

CLAUDE.md — replace `[YOUR DOMAIN]` placeholders.

Wiki entity links — `[[entities/openscaffold]]` in found pages can stay or be replaced.

he list in `raw/` — replace with your own

Keep as-is: all six skills, generic operational tools, vault directory structure.

Don't delete the mandatory-first-action bar the end-of-session checklist in CLAUDE.m

---

The Limitless Stack is one integrated system. Don't try to use just one tool — the value compounds across all seven, across every session, across every bug fix and every source ingest. The stack you run on day 90 won't be the same stack you booted on day 1. That's the point.

Open Scaffold Labs · github.com/Open-Scaffold-Labs/LimitlessStack · v1.0